

Sequence Modeling: From RNNs to Mamba (SSM Perspective)

Based on Lectures 14, 16, and 17: Control Theory Concepts in Deep Learning

1 The Big Picture: Evolution of Sequence Models

The central idea is to re-frame traditional Recurrent Neural Networks (RNNs) as a specific type of signal processing model called a **State-Space Model (SSM)** or **Infinite Impulse Response (IIR) Filter** (Lec 14).

- **RNN $\rightarrow$ SSM Analogy:** The hidden state update ($\vec{h}_t$) in an RNN is structurally equivalent to the state update in a control theory SSM, performing **causal sequence modeling**.
- **Evolution Goal:** Transition from the slow, non-linear RNN to a fast, parallelizable architecture that retains the RNN's causal memory, culminating in the Selective SSM (Mamba).

2 Kalman Filter and RNN Training

The Kalman Filter (KF) is used as an **ideal reference model** for optimal state estimation. The RNN's goal is to learn the essence of this optimality.

Role of the Hidden State ($\vec{h}_t$)

The training process forces the network to learn parameters that make the hidden state $\vec{h}_t$ an **optimal, minimal, and predictive representation** of the entire input history ($\vec{x}_1, \dots, \vec{x}_t$).

Self-Supervision (Next-Thing Prediction)

The model is trained to predict the next input element, using the data itself as the label.

- **SSM Equations:**

$$\text{State Update: } \vec{h}_t = A\vec{h}_{t-1} + B\vec{x}_t$$

$$\text{Output: } \vec{y}_t = C\vec{h}_t + D\vec{x}_t$$

- **Training Objective:** Minimize the loss $\mathcal{L}$ between the predicted output $\vec{y}_t$ and the true next input $\vec{x}_{t+1}$.

$$\mathcal{L} = \sum_{t=1}^T \text{Loss}(\vec{y}_t, \vec{x}_{t+1})$$

3 The Problem with Parallelization and the Linear SSM Fix

The RNN Bottleneck: Sequential Dependency

- **Problem:** Traditional RNNs are impossible to parallelize during training.
- **Cause:** The **non-linear recurrent dependency chain** (Lec 16) where $\vec{h}_t$ depends on $\vec{h}_{t-1}$ prevents simultaneous calculation across time steps T .

$$\vec{h}_t = \sigma(A\vec{h}_{t-1} + B\vec{x}_t)$$

The Linear SSM (LSSM) Solution

- **Key Step:** Eliminate the non-linearity (σ) to create a **Linear-Time Invariant (LTI) SSM**.
- **Parallelization Method:** An LTI-SSM can be mathematically unrolled into a single long **1D Convolution** operation.

$$\text{Sequence}(\vec{y}) = \text{Filter} * \text{Sequence}(\vec{x})$$

- **Benefit:** Convolution is inherently parallelizable (unlike recurrence) and can be computed efficiently (e.g., via FFTs), enabling fast training with $\mathcal{O}(T)$ or $\mathcal{O}(T \log T)$ complexity.

4 Mamba's Solution: The Selective State-Space Model

The LSSM is fast, but it lacks the expressiveness and long-term memory of a practical sequence model. Mamba is built on the solutions to these two problems.

Challenge 1: Long-Range Dependencies (Stability)

- **Problem:** The system is unstable, leading to **Vanishing/Exploding Gradients** (Lec 17). This occurs if the eigenvalues (λ) of the state matrix A are not close to the unit circle ($|\lambda| \neq 1$).
- **Fix (S4/Mamba):** Use **Hippo-based initialization** for matrix A . This analytical pre-training sets the eigenvalues to be near the unit circle ($|\lambda| \approx 1$), ensuring **critical damping** and stable memory retention across long sequences.

Challenge 2: Lack of Expressiveness

- **Problem:** A purely linear system is insufficient for complex tasks like language modeling.
- **Fix (Mamba's Selectivity):** Introduce **Input-Dependent Gating** by making the time-discretization step ΔT a function of the input $\vec{x}_t$.

$$\Delta T = \text{softplus}(\text{Linear}(\vec{x}_t))$$

- **Result:** The model dynamically controls its memory (gating the state). A small ΔT means high memory retention, while a large ΔT means fast forgetting. This **selective filtering** solves the expressiveness problem while retaining the fast, parallel structure.